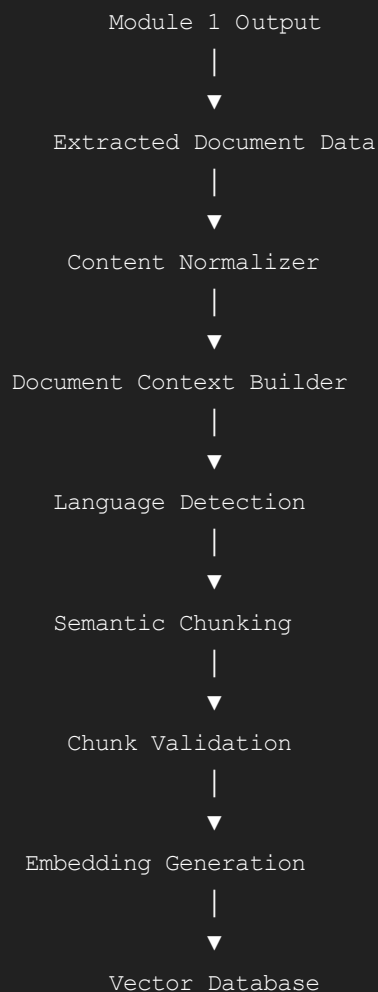


Yes. These five components are the **heart of Module 2**. They transform raw extracted text into **AI-ready knowledge**. If Module 1 is the "eyes" of Company Brain (extracting information), then Module 2 is the **brain's cortex**, where the extracted information is cleaned, understood, organized, and prepared for semantic search.

A production-grade Company Brain should not simply split text into chunks. Each stage exists for a specific reason.

---

## Module 2 Pipeline



Let's understand every stage in detail.

---

### 1. Content Normalizer

#### Purpose

The Content Normalizer converts messy extracted data into **clean, consistent text**.

Think of it like washing vegetables before cooking.

If the extracted text is dirty, every later stage becomes worse.

---

## Why do we need it?

Suppose Module 1 extracts this:

```
Company      Brain

Revenue      Increased

By           42%

North
America
```

Or this:

```
Revenue:\t\t\t42%

Sales\x00Data
```

Or

```
â€œCompany Brainâ€
```

These are common extraction artifacts.

If we generate embeddings directly, the embedding model learns incorrect patterns.

---

## Input

JSON

```
{
  "page": 5,
  "text": "Company      Brain\n\nRevenue      Increased"
}
```

---

## Output

JSON

```
{
  "page": 5,
  "text": "Company Brain\nRevenue Increased"
}
```

---

## Responsibilities

A production normalizer typically performs:

### Remove extra spaces

Before

Revenue      Increased

After

Revenue Increased

---

### Fix line breaks

Before

North  
America

After

North America

---

### Remove control characters

Before

Revenue\x00Growth

After

Revenue Growth

---

## Fix Unicode

Before

â€œHelloâ€

After

"Hello"

---

## Normalize bullets

Before

•  
-  
\*

After

•

---

## Standardize punctuation

Before

Revenue.....

After

Revenue.

---

## Preserve important formatting

Never destroy:

- Tables
- Lists
- Code
- Equations
- Markdown

## Architecture

Raw Text

↓

Whitespace Cleaner

↓

Unicode Fixer

↓

Control Character Remover

↓

Bullet Normalizer

↓

Punctuation Cleaner

↓

Clean Text

## Example Code

Python

Run

```
class ContentNormalizer:

    def normalize(self, text: str) -> str:
```

```
text = self.fix_unicode(text)

text = self.remove_control_characters(text)

text = self.normalize_whitespace(text)

text = self.normalize_bullets(text)

return text
```

---

## 2. Document Context Builder

This is one of the most important components in Company Brain.

Most RAG systems completely ignore it.

---

### Why?

Suppose the extracted text is

Revenue

North America

42%

Europe

25%

Does AI know

- what is a heading?
- what is a section?
- what belongs together?

No.

---

**Context Builder reconstructs the document.**

Suppose the PDF is

Financial Report

Chapter 1

Revenue

North America

42%

Europe

25%

## The builder understands

Document

└─ Chapter 1

└─ Revenue

└─ North America

└─ Europe

## Now every paragraph knows

its

- parent heading
- section
- page
- document

---

## Metadata Produced

JSON

```
{  
  "document_id":101,  
  
  "page":5,  
  
  "section":"Revenue",  
  
  "heading":"Chapter 1",
```

"parent": "Financial Report"

This metadata dramatically improves retrieval.

---

## Why?

Suppose user asks

Show revenue information.

Instead of retrieving random paragraphs,

AI retrieves

everything

inside

Revenue section.

---

## Architecture

Normalized Text

↓

Heading Detector

↓

Hierarchy Builder

↓

Section Builder

↓

Metadata Generator

↓

Document Context Tree

---

### 3. Language Detection

Company Brain is designed for global companies.

A company may upload:

- English contracts
- Japanese manuals
- German invoices
- Spanish HR policies

The system must identify the language before embedding.

---

#### Example

Input

Bonjour

Comment allez-vous ?

Output

JSON

```
{  
  "language": "French",  
  
  "confidence": 0.99  
}
```

#### Why?

Different languages may require:

- different embedding models
  - different tokenizers
  - different OCR settings
  - different stop words
- 

#### Architecture

Normalized Text

↓

Language Detector

↓

English

Spanish

Japanese

German

Hindi

...

---

## Libraries

Common options include:

- fastText
- langdetect
- lingua-py
- CLD3

A production system usually chooses based on accuracy, speed, and supported languages.

---

## 4. Semantic Chunking

This is the most critical stage.

Most beginners split every

500 words.

That is wrong.

---

Suppose

Chapter 5

Employee Benefits

Medical Insurance

Life Insurance

Retirement Plan

Travel Allowance

A fixed chunker might split like this:

Chunk 1

Medical Insurance

Life Insurance

Chunk 2

Retirement Plan

Travel Allowance

The heading

Employee Benefits

is lost.

---

Semantic Chunking keeps ideas together.

Instead

Employee Benefits

Medical Insurance

Life Insurance

Retirement Plan

Travel Allowance

becomes

one chunk.

---

## Parent-Child Chunking

Company Brain uses

Parent



Child

Example

Parent Chunk

Revenue Analysis

contains

Child 1

North America

Child 2

Europe

Child 3

Asia

Later

AI retrieves

Child

but also knows

its Parent.

This greatly improves answer quality.

---

## Metadata

Every chunk stores

JSON

```
{
  "chunk_id":55,

  "parent_chunk":12,

  "page":6,

  "heading":"Revenue",

  "section":"North America"
}
```

---

## Architecture

Document Context Tree

↓

Paragraph Detection

↓

Semantic Split

↓

Parent Chunk

↓

Child Chunks

↓

Chunk Metadata

---

## 5. Chunk Validation

Many systems stop after chunking.

Company Brain doesn't.

It evaluates every chunk before generating embeddings.

---

## Why?

Suppose a chunk contains

-----

42

Page 7

Confidential

-----

Generating embeddings is wasteful.

The validator rejects low-value chunks.

---

## What does it check?

### Minimum length

Reject

Yes.

---

### Maximum length

Reject

very large chunks.

---

### Structural quality

Is it mostly

- text?
- numbers?

- symbols?
- 

## Duplicate detection

Suppose

same paragraph

appears

20 times.

Keep one.

---

## Empty chunks

Reject

""

---

## OCR confidence

If OCR confidence

is

20%

↓

Ignore

or flag

for review.

---

## Business importance

Suppose

Revenue

42%

is likely more valuable than

Page 12

The validator can assign an **importance score** based on business rules.

## Output

JSON

```
{
  "chunk_id":55,

  "valid":true,

  "importance":0.94,

  "reason":"High quality"
}
```

## Complete Module 2 Flow

Module 1

(Extracted Text)



Content Normalizer

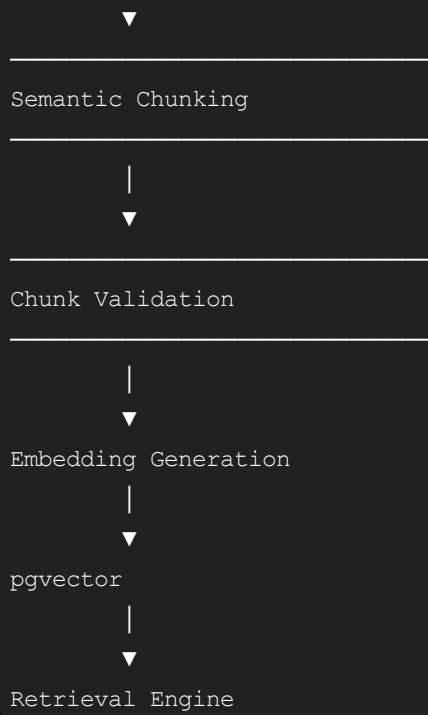


Document Context Builder



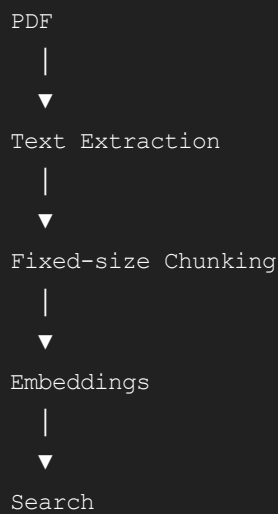
Language Detection



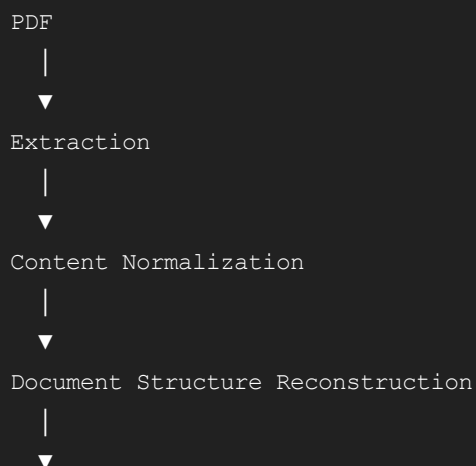


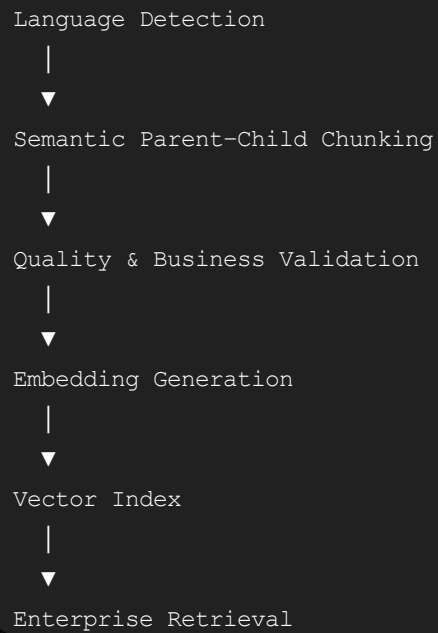
## Why this pipeline is a competitive advantage

Many RAG systems follow a simple pipeline:



Company Brain's pipeline is much richer:





This preserves document structure, improves retrieval precision, reduces unnecessary embeddings, supports multilingual content, and gives AI agents the context they need to produce more accurate answers. It's one of the key differences between a basic document chatbot and an enterprise-grade "Company Brain."